Phase 6 | System Design and Scalability

Explain current data paths, scaling constraints, and a practical evolution path.

React/Vite Edition | Repository-grounded study material | Snapshot: 2026-10-24

1. Current architecture

The main browser app is a Vite-built React SPA. It calls Express for account routes, alerts, nearby search and uploads. Express uses MongoDB Atlas and selected external feeds. The chatbot is a separate Flask deployment that calls Gemini. Vercel configuration rewrites API traffic to the Express host; Render runs the backend. A single Node deployment can also serve the frontend build.

2. Core request paths

Flow	Sequence
Sign-in	React form -> Express auth route -> Mongoose User lookup/create -> bcrypt compare/hash -> JWT response.
Google OAuth	Browser -> Passport authorize -> Google -> callback -> user record -> app JWT -> frontend redirect.
Nearby	Browser location -> React fetch -> Express validation/cache -> Geoapify Places -> normalized JSON -> list and Leaflet markers.
Alert detail	Ticker -> Express summary -> SACHET feed -> summary JSON -> detail route fetches CAP XML -> parser -> readable view.
Chat	React chat -> Flask /chat -> Gemini API -> extracted response JSON -> chat UI.

3. Current scaling properties

Mechanism	Current property	Scaling concern
In-memory nearby cache	Fast and process-local, with stale window.	Each instance has a separate cache; restart clears it.
MongoDB Atlas	Managed persistence separate from web workers.	Use bounded connections, indexes, backups and least-privilege access.
Express	Can run stateless instances behind a proxy.	Sessions/cache should be externalized if shared state becomes necessary.
Vercel static assets	Frontend assets are served separately from API compute.	Rewrites and API availability remain coupled to the Render backend.
Flask chatbot	Separately deployable and scalable.	Needs rate limits, request size bounds and provider quota controls.
External APIs	Provider latency and quotas are external dependencies.	Cache, timeouts, retries and fallback UX are essential.

4. Reliability improvements

- Use request timeouts for every upstream HTTP call and return controlled errors.
- Track provider freshness and preserve last-known-good results where suitable.
- Use request coalescing for identical in-flight requests; shared cache only when multiple instances need it.
- Expose health/readiness separately from liveness so database failures are visible.
- Add structured request IDs, latency metrics and provider status counts.
- Rate-limit authentication, chatbot and expensive provider calls.

5. Deployment evolution

For a small deployment, keep the frontend static and Express API separate, but document environment variables and service URLs in one source of truth. For higher traffic, add an API gateway or stable reverse proxy, shared Redis cache, central logging, secret manager and background ingestion for alert feeds. Keep AI provider calls isolated and budgeted. Avoid adding infrastructure until traffic, reliability or coordination needs justify the cost.

Scalability answer: first make each service stateless and observable, then move process-local state to shared infrastructure only when multiple instances require it.